Explain why retaining a two-worker barrier inside the mutex-protected increment could deadlock.

- A barrier inside the mutex-protected increment could lead to a situation where the first worker to hold the mutex lock while waiting for the second worker to arrive, but the second worker is locked out and unable to reach the barrier, so the first worker waits forever.

Provide a numbered trace of the forced run showing each read and write, private values, and final counter.

-
```
_Forced race condition_:
Event 1: Worker 1 READ private counter = 0
Event 2: Worker 2 READ private counter = 0
Event 3: Worker 2 WRITE private counter = 1
Event 4: Worker 1 WRITE private counter = 1
Final counter after forced run: 1
```

Provide the repaired run and explain why its final counter must be 2 after both workers finish.

-
```
_Mutex repair_:
Event 1: Worker 1 READ private counter = 0
Event 2: Worker 1 WRITE private counter = 1
Event 3: Worker 2 READ private counter = 1
Event 4: Worker 2 WRITE private counter = 2
Final counter after repaired run: 2
```

- Each worker reads the latest value, increments it, and writes it before releasing the lock. Since only one worker can read-write-modify the counter at a time, the counter increments to 1 after worker 1 and 2 after worker 2.

Explain why individually atomic loads/stores did not make the original full increment atomic.

- An atomic load/store ensures that reading/writing the counter is safe, respectively, but it doesn't make the full sequence of reading, modifying, and writing atomic

Point to the exact gates and locks in your source. State why the forced experiment guarantees its observed final value.

- In the forced worker function, I use pthread_barrier_wait(&read_barrier) to force both of the workers to finishing reading before they can increment or write. This is why we end up with a final value of 1, the barrier forces incorrect execution by forcing the workers to wait for each other, so it is guaranteed that the workers will end up in a race condition where they both read and increment zero, and write back 1, no matter which one reads first.
- In the repaired execution, I used a mutex lock around the read-increment-write block: pthread_mutex_lock(&counter_mutex); and pthread_mutex_unlock(&counter_mutex); This way, only one worker can read-update-write to te counter at a time, and must wait for the other worker to release the lock before they can access and work on the counter.